WEEK 3: ANALYTICAL MODELLING (POSTGRESQL)

Based on Kimball & Ross, "The Data Warehouse Toolkit", 3rd Edition

Who is Ralph Kimball?

Ralph Kimball (b. 1944) is the father of dimensional modelling.

- **1996** — Published *The Data Warehouse Toolkit* (1st ed.)
- **2013** — 3rd edition with Margy Ross: still the industry reference
- Founded the **Kimball Group**: training & consulting in DW/BI design
- Over **450,000 copies** of the Toolkit sold worldwide

> "The data warehouse must present information so that it is accessible and understandable to the business community."
> — Ralph Kimball

Kimball's approach is also called **bottom-up**: build data marts first, integrate later via conformed dimensions.

OLTP vs OLAP

OLTP: Online Transaction Processing

- Purpose: Run the business (insert, update, delete)
- Example: POS system, e-commerce checkout, bank ATM
- Normalised schema (3NF) — avoids redundancy
- Row-oriented access — optimised for single-row writes
- Focus: current state, fast writes, data integrity

OLAP: Online Analytical Processing

- Purpose: Analyse the business (read, aggregate, time-series)
- Example: Sales dashboards, customer trend reports, margin analysis
- **Denormalised** schema — redundancy is acceptable for speed
- Column-oriented access — optimised for aggregation over many rows
- Focus: historical trends, fast reads, business questions

	OLTP	OLAP
Operations	INSERT / UPDATE	SELECT / GROUP BY
Schema	Normalised (3NF)	Dimensional (star)
Users	Applications	Analysts / Dashboards
Data	Current	Historical

This week: OLAP. Building **data warehouses** using Kimball's method.

Why Kimball? The Case for Dimensional Modelling

1. Business users can read it

- Star schemas mirror how business people think: "sales by product by month"
- An analyst can navigate the schema without database expertise

2. BI tools are built for it

- Tableau, Power BI, Looker, MicroStrategy all auto-detect star schemas
- Drag-and-drop from dimension attributes directly onto reports
- No custom SQL needed for standard reports

3. Consistent, predictable query patterns

- All analytical queries follow the same template: JOIN dims to fact, GROUP BY, SUM
- Easy to optimise with indexes and aggregates

4. Incrementally buildable

- Start with one business process (e.g., sales)
- Add more (inventory, HR) using **conformed dimensions** — no redesign required

5. Proven at scale

- Used in retail, healthcare, finance, government for 30+ years
- Works with traditional RDBMS, columnar stores (Redshift, BigQuery, Snowflake), and OLAP cubes

6. Industry-standard vocabulary

- "Fact", "Dimension", "Grain", "Conformed" — universal language across teams

> Kimball's approach directly answers the question: *"How do I build a database that makes BI tools sing?"*

Kimball's Four-Step Design Process

Kimball defines a **four-step process** for every dimensional model you design.

These must be answered **in order**.

Step 1 — Select the Business Process

A **business process** is a business activity that generates measurable events.

Examples:

- **Retail:** Point-of-sale transactions
- **Healthcare:** Patient admissions
- **Finance:** Trade executions
- **Web:** User clickstream events

> Do NOT model org charts or departments. Model **what the business does**.

Each business process becomes one or more **fact tables** in your warehouse.

Step 2 — Declare the Grain

Grain = *"What does one row in the fact table represent?"*

This is the **most critical decision** in dimensional modelling.

Grain Choice	Description
Individual line item on a receipt	Most atomic — maximum flexibility
Daily totals per store per product	Pre-aggregated — less storage, less flexibility
One row per customer per month	Snapshot — for periodic analysis

Kimball's rule: *Declare the grain before identifying dimensions or facts.*

Once set, every fact column must be true to the grain.

Step 3 — Identify the Dimensions

Dimensions provide the **context** for each measurement.

Ask: *"Who, what, when, where, why?"*

Question	Dimension
When?	**Date / Time**
What?	**Product**
Where?	**Store / Location**
Who?	**Customer**
Why?	**Promotion**

> Dimensions are the **entry points** into the fact table. Every filter or GROUP BY in an analytical query is a dimension attribute.

Step 4 — Identify the Facts

Facts are the **measurements** captured at the declared grain.

Ask: *"What are we measuring, and is it additive?"*

Measure type	Can SUM?	Example
Additive	Across all dimensions	sales_amount, quantity
Semi-additive	Some dimensions only	account_balance (not time)
Non-additive	Never	unit_price, percentage

> Kimball: *"Facts are almost always numeric and additive. If it is not additive, question whether it belongs in the fact table."*

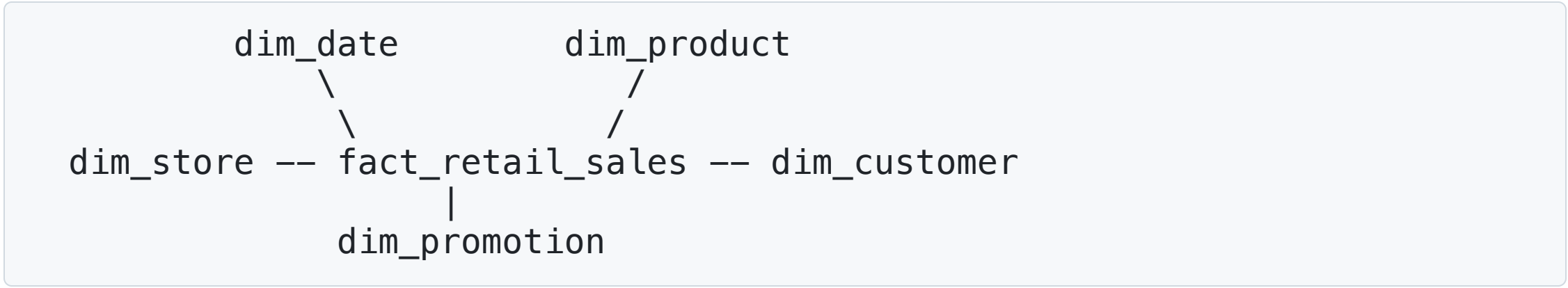
Kimball's Canonical Example: Retail Sales

Drawn directly from **Chapter 3 of The Data Warehouse Toolkit (3rd Ed.).**

Business process: Point-of-sale retail transactions

Grain: One row per **line item** on a sales receipt

This is the most-cited example in dimensional modelling literature.



Five tables. Every analyst in the world recognises this pattern.

Star Schema

A **star schema** has:

- **One central fact table** — the measurements
- **Dimension tables** radiating out — the context

The name comes from the visual layout (it looks like a star).

Why not fully normalise?

- A normalised product table might split into 5 sub-tables (brand, category, supplier...)
- Every query would need 5 extra JOINs — slow and complex
- Kimball's answer: **flatten dimensions** — accept redundancy, gain speed

> "Simplicity is the most important property of a dimensional model."
> — Kimball & Ross, DW Toolkit 3rd Ed.

Fact Table Structure (Retail Sales)

```
CREATE TABLE fact_retail_sales (
    sales_key              SERIAL        PRIMARY KEY,
    -- Foreign keys to all dimensions
    date_key               INTEGER       NOT NULL REFERENCES dim_date(date_key),
    product_key            INTEGER       NOT NULL REFERENCES dim_product(product_key),
    store_key              INTEGER       NOT NULL REFERENCES dim_store(store_key),
    customer_key           INTEGER       NOT NULL REFERENCES dim_customer(customer_key),
    promotion_key          INTEGER       NOT NULL REFERENCES dim_promotion(promotion_key),
    -- Degenerate dimension: receipt number (no separate table needed)
    pos_transaction_number VARCHAR(20),
    -- Additive measures
    sales_quantity         INTEGER       NOT NULL,
    unit_price             NUMERIC(10,2) NOT NULL,
    sales_amount           NUMERIC(10,2) NOT NULL,
    cost_amount            NUMERIC(10,2) NOT NULL,
    gross_profit_amount    NUMERIC(10,2) NOT NULL
);
```

Degenerate dimension: `pos_transaction_number` — an operational key stored on the fact with no dimension table (Kimball DW Toolkit Ch. 3).

Fact Table Types (Kimball)

Kimball defines **three types** of fact tables:

1. **Transaction fact table** (most common)
   - One row per discrete event (sale, click, payment)
   - Most atomic grain — maximum flexibility
   - Example: `fact_retail_sales` (one row per line item)
2. **Periodic snapshot fact table**
   - One row per period (day, week, month) per entity
   - Example: daily inventory levels, monthly account balance
   - Captures state at a point in time

3. Accumulating snapshot fact table

- One row per entity across its entire lifecycle
- Updated as the process progresses (order → shipped → delivered)
- Example: order fulfilment pipeline

The Date Dimension

Kimball calls the date dimension **"the most important dimension in the warehouse"**.

Key Kimball rules for dim_date:

1. Pre-populate it entirely at build time (no live joins to a calendar)
2. Use an **integer surrogate key** in YYYYMMDD format
3. Include every business-relevant attribute, not just date and month

```sql
CREATE TABLE dim_date (
    date_key          INTEGER     PRIMARY KEY,  -- e.g. 20240115
    full_date         DATE        NOT NULL,
    day_of_week       VARCHAR(10),  -- 'Monday', 'Tuesday'...
    calendar_month    VARCHAR(10),  -- 'January', 'February'...
    month_num         SMALLINT,
    calendar_quarter  CHAR(2),      -- 'Q1', 'Q2'...
    calendar_year     SMALLINT,
    is_weekend        BOOLEAN
);
```

Why YYYYMMDD integer? Human-readable, sortable, partition-friendly, and fast for range scans.

Populate dim_date with generate_series:

```
INSERT INTO dim_date (
    date_key, full_date, day_of_week, calendar_month,
    month_num, calendar_quarter, calendar_year, is_weekend
)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INTEGER          AS date_key,
    d::DATE                                   AS full_date,
    TO_CHAR(d, 'Day')                         AS day_of_week,
    TO_CHAR(d, 'Month')                       AS calendar_month,
    EXTRACT(MONTH   FROM d)::SMALLINT         AS month_num,
    'Q' || EXTRACT(QUARTER FROM d)::TEXT      AS calendar_quarter,
    EXTRACT(YEAR    FROM d)::SMALLINT         AS calendar_year,
    EXTRACT(DOW     FROM d) IN (0, 6)         AS is_weekend
FROM generate_series(
    '2024-01-01'::DATE,
    '2024-03-31'::DATE,
    '1 day'
) AS d;
```

Dimension Tables

A **dimension table** contains all the descriptive attributes for one axis of analysis.

Kimball's rules for dimension tables:

1. Use a **surrogate key** as the primary key (not the business key)
2. Denormalise — flatten hierarchies into a single wide table
3. Use real words for column names, not codes

```sql
CREATE TABLE dim_product (
    product_key         SERIAL        PRIMARY KEY,   -- surrogate key
    product_sku         VARCHAR(20)   NOT NULL,       -- natural / operational key
    product_description VARCHAR(255),
    brand               VARCHAR(100),
    category            VARCHAR(100),  -- e.g. 'Computers'
    department          VARCHAR(100),  -- e.g. 'Electronics' (hierarchy flattened)
    package_type        VARCHAR(50),
    is_low_fat          BOOLEAN        DEFAULT FALSE,
    is_recyclable       BOOLEAN        DEFAULT FALSE
);
```

Note: `department -> category -> product` is a **hierarchy embedded as flat columns** — the Kimball way.

The store dimension with a geographic hierarchy:

```
CREATE TABLE dim_store (
    store_key    SERIAL        PRIMARY KEY,
    store_name   VARCHAR(100) NOT NULL,
    store_number VARCHAR(20)  NOT NULL,
    district     VARCHAR(100) NOT NULL,
    region       VARCHAR(100) NOT NULL,  -- hierarchy: store -> district -> region
    country      VARCHAR(100) NOT NULL,
    total_sq_ft  INTEGER,
    open_date    DATE
);
```

Analytic payoff:

```
-- Group by any level of the hierarchy — no extra joins needed
SELECT region, SUM(sales_amount)
FROM fact_retail_sales f
JOIN dim_store s ON f.store_key = s.store_key
GROUP BY region;
```

Surrogate Keys

Surrogate key = auto-generated artificial integer PK, independent of source systems.

Why Kimball insists on surrogate keys:

Reason	Explanation
Stability	Business keys change (email, SKU); surrogate keys never do
Small foreign keys	INT is 4 bytes; a compound business key might be 100+ bytes
SCD support	Each version of a dimension row gets a unique surrogate
Source integration	Two source systems with the same natural key? No conflict

```
-- Surrogate key in the fact table is just an integer
customer_key  INTEGER  REFERENCES dim_customer(customer_key)

-- If Alice changes her email, only dim_customer is updated.
-- fact_retail_sales keeps pointing to the same customer_key.
```

> Kimball: *"The surrogate key is the glue between the fact and dimension tables."*

Conformed Dimensions & the Bus Architecture

Conformed dimension = a dimension shared across multiple fact tables.

```
                  dim_date    dim_product
                     |            |
                     v            v
fact_retail_sales ------------------
fact_inventory    ------------------
                     ^            ^
                  dim_date    dim_product
```

`dim_date` is the same table in both. Query both, compare results — this is **drilling across**.

Enterprise Data Warehouse Bus Architecture (Kimball):

> "An incremental approach to building the enterprise DW/BI system. Integration is achieved via standardised conformed dimensions reused across business processes."

The **Bus Matrix** maps business processes (rows) to dimensions (columns).

Bus Matrix example (retail):

Business Process	Date	Product	Store	Customer	Promotion
Retail Sales	Y	Y	Y	Y	Y
Inventory	Y	Y	Y		
Web Orders	Y	Y		Y	Y

The shared Y columns are conformed dimensions.

- One `dim_date` table serves all three fact tables
- One `dim_product` table serves Sales, Inventory, and Web Orders

This is how Kimball's warehouse grows **incrementally** — each new business process reuses existing dimensions.

Why Kimball Works Beautifully with BI Tools

BI tools like **Tableau**, **Power BI**, **Looker**, and **Metabase** are designed around the star schema pattern.

Power BI / Tableau:

- Auto-detects relationships between fact and dimension tables
- Each dimension attribute becomes a draggable field
- Measures (SUM, AVG) are defined once and reused across all reports

Why it works:

- BI tools generate SQL like `SELECT dim_col, SUM(fact_col) ... JOIN ... GROUP BY`
- This is **exactly** the query pattern star schemas are optimised for
- Indexed FK columns on the fact table + small dimension tables = fast response

Example: Power BI / Tableau auto-generated query pattern

```sql
-- BI tools generate this pattern automatically from a star schema:
SELECT
    p.category              AS product_category,
    d.calendar_quarter      AS quarter,
    SUM(f.sales_amount)     AS total_sales
FROM fact_retail_sales f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_date    d ON f.date_key    = d.date_key
GROUP BY p.category, d.calendar_quarter
ORDER BY d.calendar_quarter, p.category;
```

A normalised OLTP schema would require many more JOINs and the BI tool's query generator would struggle — or produce incorrect results.

Slowly Changing Dimensions (SCD)

Dimensions change over time. Kimball defines **eight SCD types** (0-7).

The three most important:

Type 0 — Retain Original

- Attribute never changes once loaded
- Example: customer's date of birth

Type 1 — Overwrite (lose history)

- Update the attribute in place; history is lost
- Example: correcting a data entry error

Type 2 — Add New Row (full history — the Kimball default)

- Expire the old row, insert a new row with updated value
- Every historical fact still points to the correct version of the dimension
- Example: customer moves city; product changes category

SCD Type 2 — Implementation (Kimball Standard)

Add three columns to the dimension table:

```
CREATE TABLE dim_customer (
    customer_key         SERIAL       PRIMARY KEY,  -- surrogate (new per version)
    customer_durable_id  VARCHAR(20)  NOT NULL,      -- natural key (stable)
    full_name            VARCHAR(255),
    city                 VARCHAR(100),
    segment              VARCHAR(50),
    -- SCD Type 2 tracking (Kimball DW Toolkit, Ch. 5)
    effective_date       DATE    NOT NULL DEFAULT CURRENT_DATE,
    expiry_date          DATE    NOT NULL DEFAULT '9999-12-31',
    is_current           BOOLEAN NOT NULL DEFAULT TRUE
);
```

SCD Type 2 Example: Before and After

BEFORE: Alice (C001) lives in London

```
 customer_key | customer_durable_id | full_name   | city   | effective_date | expiry_date | is_current
--------------+---------------------+-------------+--------+----------------+-------------+-----------
            1 | C001                | Alice Brown | London | 2024-01-01     | 9999-12-31  | TRUE
```

When **Alice moves from London to Bristol** on 2024-02-01:

```sql
-- Step 1: expire the old record
UPDATE dim_customer
SET expiry_date = '2024-01-31', is_current = FALSE
WHERE customer_durable_id = 'C001' AND is_current = TRUE;

-- Step 2: insert the new version
INSERT INTO dim_customer (customer_durable_id, full_name, city, segment,
                          effective_date, expiry_date, is_current)
VALUES ('C001', 'Alice Brown', 'Bristol', 'Premium', '2024-02-01', '9999-12-31', TRUE);
```

AFTER: Alice now has two rows — one historical, one current

Query current and historical states:

```sql
-- Current customers only
SELECT customer_durable_id, full_name, city
FROM dim_customer
WHERE is_current = TRUE;

-- What city did Alice live in on 2024-01-15?
SELECT customer_durable_id, full_name, city
FROM dim_customer
WHERE customer_durable_id = 'C001'
  AND '2024-01-15' BETWEEN effective_date AND expiry_date;
```

Result of the second query:

```
 customer_durable_id |  full_name  |  city
---------------------+-------------+--------
 C001                | Alice Brown | London
```

Key insight: Historical facts in `fact_retail_sales` pointing to the old `customer_key`

Database Indexes

An **index** is a data structure that lets the database skip full table scans.

Think of a book index: instead of reading every page, jump to the right section.

Without index on customer_key:

```
fact_retail_sales: 10,000,000 rows
Query: WHERE customer_key = 1001
Result: scan all 10 million rows (sequential scan)
```

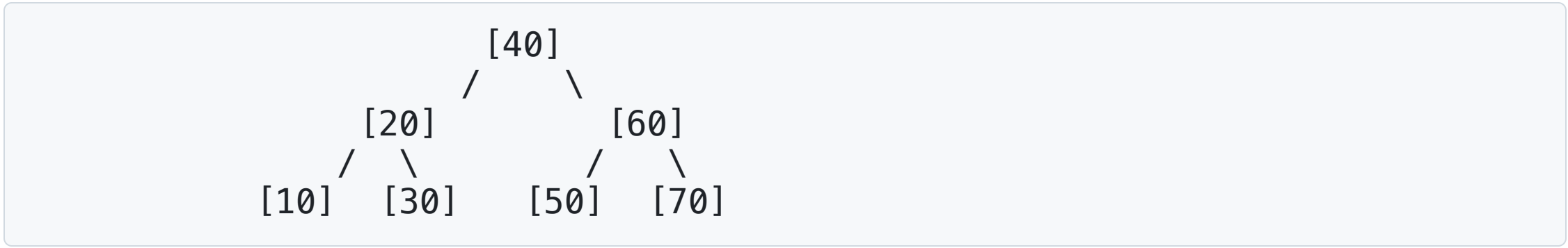
With index on customer_key:

```
INDEX on customer_key: B-tree with 10M entries
Query: WHERE customer_key = 1001
Result: ~23 comparisons (log2 10M ~ 23) then direct access
```

Kimball explicitly recommends indexing **all foreign key columns** in fact tables.

B-Tree Indexes

B-Tree = Balanced Tree — the default index type in PostgreSQL.



Properties:

- Always balanced — lookup depth is O(log n)
- Sorted — enables range queries without scanning the whole index
- Self-balancing — inserts/deletes maintain structure

Supports:

- Equality: `WHERE product_key = 3`
- Range: `WHERE date_key BETWEEN 20240101 AND 20240331`
- Prefix: `WHERE product_sku LIKE 'SKU-1%'`
- Sort: `ORDER BY date_key`

When to Index — Kimball's Guidance

Always index in a star schema:

```
-- All FK columns in the fact table (Kimball DW Toolkit, Ch. 19)
CREATE INDEX idx_fact_date     ON fact_retail_sales(date_key);
CREATE INDEX idx_fact_product  ON fact_retail_sales(product_key);
CREATE INDEX idx_fact_store    ON fact_retail_sales(store_key);
CREATE INDEX idx_fact_customer ON fact_retail_sales(customer_key);

-- Frequently-filtered dimension attributes
CREATE INDEX idx_product_category ON dim_product(category);
CREATE INDEX idx_customer_segment ON dim_customer(segment);
CREATE INDEX idx_store_region     ON dim_store(region);
```

Do NOT index:

- Low-cardinality columns: `is_weekend` (only 2 values — full scan is faster)
- Rarely-queried columns
- Write-heavy staging tables (indexes slow down bulk loads)

CASE Expressions

CASE = conditional logic inside SQL. Essential for BI and data warehouse work.

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

Example: classify orders by value tier

```
SELECT
    pos_transaction_number,
    sales_amount,
    CASE
        WHEN sales_amount > 500 THEN 'High Value'
        WHEN sales_amount >= 50 THEN 'Mid Value'
        ELSE                         'Low Value'
    END AS value_tier
FROM fact_retail_sales;
```

CASE in Aggregations

CASE + SUM is a classic Kimball technique for multi-metric reports:

```sql
-- Count high-value vs low-value lines by product department
SELECT
    p.department,
    SUM(CASE WHEN f.sales_amount > 500 THEN 1 ELSE 0 END) AS high_value_lines,
    SUM(CASE WHEN f.sales_amount BETWEEN 50 AND 500
                                       THEN 1 ELSE 0 END) AS mid_value_lines,
    SUM(CASE WHEN f.sales_amount < 50  THEN 1 ELSE 0 END) AS low_value_lines,
    COUNT(*)                                              AS total_lines
FROM fact_retail_sales f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.department
ORDER BY total_lines DESC;
```

This produces **one row per department** with multiple metrics — exactly what a BI dashboard needs.

Analytical Query Patterns (Summary)

```sql
-- 1. Revenue and margin by product category
SELECT p.category,
       SUM(f.sales_amount)        AS revenue,
       SUM(f.gross_profit_amount) AS gross_profit
FROM fact_retail_sales f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.category ORDER BY revenue DESC;

-- 2. Sales by customer segment and quarter
SELECT c.segment, d.calendar_quarter, SUM(f.sales_amount) AS revenue
FROM fact_retail_sales f
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_date d     ON f.date_key     = d.date_key
WHERE c.is_current = TRUE
GROUP BY c.segment, d.calendar_quarter;

-- 3. Promotion effectiveness
SELECT pr.promotion_name, SUM(f.sales_amount) AS promoted_revenue
FROM fact_retail_sales f
JOIN dim_promotion pr ON f.promotion_key = pr.promotion_key
GROUP BY pr.promotion_name ORDER BY promoted_revenue DESC;
```

Summary

Week 3 covered:

- **OLTP vs OLAP** — transactional vs analytical workloads
- **Who is Kimball** — the origin of dimensional modelling
- **Why Kimball** — business-readable, BI-tool-friendly, incrementally buildable
- **Four-Step Process** — Business Process → Grain → Dimensions → Facts
- **Star schema** — one fact table, multiple dimension tables

- **Fact table types** — transaction, periodic snapshot, accumulating snapshot
- **The Date dimension** — Kimball's most important dimension
- **Surrogate keys** — stability and SCD support
- **Conformed dimensions + Bus Architecture** — enterprise integration
- **SCD Type 2** — full history with effective/expiry dates
- **Indexes** — FK columns on fact tables, selective dimension attributes
- **CASE expressions** — conditional logic and multi-metric aggregation

You can now:

- Apply Kimball's four-step design process to any business problem
- Build a star schema that BI tools can connect to out of the box
- Handle slowly changing dimensions with Type 2 correctly
- Choose the right indexes for analytical queries
- Write CASE-based aggregation queries for dashboards

Reference: Kimball & Ross, *The Data Warehouse Toolkit*, 3rd Edition (2013)

Next week:

- NoSQL thinking (MongoDB)
- Document-oriented design
- Aggregation pipelines

EXERCISES

All exercises run against the `demo` database.

Use: `docker exec -i postgres16 psql -U postgres -d demo < code/week3.sql`

Exercise 1: Apply the Four-Step Process

Scenario: A UK retail chain wants to analyse its point-of-sale data.

Apply Kimball's four-step process:

1. **Business process** — what activity are we modelling?
2. **Grain** — what does one row in the fact table represent?
3. **Dimensions** — what context columns surround the fact? (who/what/when/where/why)
4. **Facts** — what numeric measures are captured at the grain?

The schema we built (`fact_retail_sales`) follows Kimball's retail example.
Verify your answers match the table definition in `code/week3.sql`.

Your task:

Document your answers for each of the four steps. Compare your dimensional model design with the implemented schema in `week3.sql`.

Consider:

- Why did we choose line-item grain rather than transaction-level grain?
- What would change if we used daily aggregate grain instead?
- Are all the dimensions necessary? Could any be optional?

Exercise 2: Build and Query the Warehouse

The schema is already created by `week3.sql`. Run these queries:

Query 1: Total revenue and gross profit by product department

```sql
SELECT
    p.department,
    SUM(f.sales_amount)        AS total_revenue,
    SUM(f.gross_profit_amount) AS total_gross_profit
FROM fact_retail_sales f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.department
ORDER BY total_revenue DESC;
```

Query 2: Sales by customer segment

```
SELECT
    c.segment,
    COUNT(DISTINCT f.pos_transaction_number) AS num_transactions,
    SUM(f.sales_amount)                      AS total_revenue,
    ROUND(AVG(f.sales_amount), 2)            AS avg_line_item_value
FROM fact_retail_sales f
JOIN dim_customer c ON f.customer_key = c.customer_key
WHERE c.is_current = TRUE AND c.customer_key != 0
GROUP BY c.segment
ORDER BY total_revenue DESC;
```

Expected results:

- Electronics department should show highest revenue
- Premium segment customers should show higher average transaction values
- You should see transactions from all four stores

Exercise 2: Build and Query the Warehouse (Page 2)

Your task:

1. Run both queries and interpret the results
2. Modify Query 1 to add a `gross_margin_pct` column
3. Modify Query 2 to show only Premium segment customers
4. Write a new query combining store region and product department

Challenge: Can you write a query that shows revenue by store AND product category in a single result set?

Exercise 3: CASE Expressions for BI Metrics

Query 3: High-value vs low-value lines per department

```
SELECT
    p.department,
    SUM(CASE WHEN f.sales_amount > 500 THEN 1 ELSE 0 END) AS high_value_lines,
    SUM(CASE WHEN f.sales_amount BETWEEN 50 AND 500
                                    THEN 1 ELSE 0 END) AS mid_value_lines,
    SUM(CASE WHEN f.sales_amount < 50  THEN 1 ELSE 0 END) AS low_value_lines
FROM fact_retail_sales f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.department;
```

Query 4: Weekend vs weekday revenue using the date dimension

```
SELECT
    CASE WHEN d.is_weekend THEN 'Weekend' ELSE 'Weekday' END AS day_type,
    COUNT(*)              AS num_line_items,
    SUM(f.sales_amount) AS total_revenue
FROM fact_retail_sales f
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY d.is_weekend;
```

Expected results:

- Each department should show different distributions of value tiers
- Weekend vs weekday revenue split should show different patterns

Exercise 3: CASE Expressions for BI Metrics

Your task:

1. Run both queries and analyze the distribution patterns
2. Create a CASE expression to classify products as "Tech" vs "Non-Tech"
3. Write a query using CASE to flag "high-profit-margin" products (>40%)
4. Combine multiple CASE expressions to create a customer value segmentation

Challenge: Use CASE with date functions to create seasonal categories (Winter, Spring, Summer, Fall).

Exercise 4: SCD Type 2 Implementation

Alice (C001) moves from London to Bristol on 2024-02-01.

The steps are in `week3.sql` (Step 5). After running them, answer:

1. How many rows exist for customer `C001` in `dim_customer`?
2. Which row has `is_current = TRUE`?
3. Run this query — which city does it return, and why?

```sql
SELECT customer_durable_id, full_name, city, effective_date, expiry_date
FROM dim_customer
WHERE customer_durable_id = 'C001'
  AND '2024-01-15' BETWEEN effective_date AND expiry_date;
```

Expected result: London (because Jan 15 falls within the London period)

4. What happens to the January sales facts for Alice?
   Do they now show London or Bristol? Why?

Your task:

- Verify that historical facts remain unchanged
- Write a query to show all of Alice's purchases with the correct city for each date
- Understand why the surrogate key approach preserves history

Challenge: Design an SCD Type 2 solution for a product that changes category. What columns would you need? Write the UPDATE and INSERT statements.

Exercise 5: Design the Bus Matrix

Scenario: The retail chain wants to build two more fact tables:

- `fact_inventory` — daily stock levels per product per store
- `fact_web_orders` — online orders per product per customer

Complete the Bus Matrix:

Business Process	Date	Product	Store	Customer	Promotion
Retail Sales	Y	Y	Y	Y	Y
Inventory	?	?	?	?	?
Web Orders	?	?	?	?	?

- Which dimensions are **conformed** (shared across processes)?
- Which dimensions are **local** to one process only?
- What is the grain of each new fact table?

Hints:

- Inventory has no customer or promotion
- Web orders have no physical store
- Think about which dimensions make business sense for each process

Your task:

1. Fill in the Bus Matrix with Y (yes) or blank
2. Identify which dimensions can be reused (conformed)
3. Determine if any new dimensions are needed
4. Sketch the ERD for `fact_inventory`

Challenge: Design a third fact table for `fact_shipments` (orders → shipped → delivered lifecycle). What type of fact table is this? What additional date columns would you need?

Exercise 6: Reflection

1. What is the grain of `fact_retail_sales`? (Complete: "One row per...")
2. Why use a YYYYMMDD integer for `date_key` instead of a `DATE` column?
3. `fact_retail_sales` has five indexes on FK columns. What would happen to query performance if you removed all five indexes on a table with 10 million rows?
4. A product moves from the "Electronics" department to a new "Technology" department. Should you use SCD Type 1 or Type 2? Justify your answer with reference to historical reporting requirements.

Exercise 6: Reflection

5. Design a dimensional model for a **hospital**:
    - Step 1: Select the business process (e.g., patient admissions, surgeries, appointments)
    - Step 2: Declare the grain (e.g., one row per…?)
    - Step 3: Identify at least four dimensions (think: who, what, when, where)
    - Step 4: List three additive measures for the fact table (numeric, summable)

Consider: How would you handle patient readmissions? What about diagnosis codes that change over time?